

P1377R0

Summary of Dec 2018 SG14 freestanding discussions

Published Proposal, 2019-01-12

This version:

<http://wg21.link/P1377R0>

Author:

[Ben Craig](#) (National Instruments)

Audience:

WG21

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/ben-craig/freestanding_proposal/blob/master/sg14_recap.bs

Abstract

Summary of discussion in Dec 2018 SG14 telecon

Table of Contents

- 1 Summary**
- 2 Opening commentary on subsetting**
- 3 Lead in to presentation**
- 4 Presentation material from Ben Craig**
 - 4.1 "What is embedded?" is the wrong question.
 - 4.2 "What is an OS?" not useful either.

- 4.3 What is a standard?
- 4.4 Conforming C++ hosted toolchain experience
- 4.5 Hosted C++: The dark side
- 4.6 Freestanding C++17 experience: Theory
- 4.7 Freestanding C++17 experience: Practice
- 4.8 Freestanding C++ experience: Dark Side
- 4.9 Source of the bits?
- 4.10 Freestanding Goal
- 4.11 Problem features in priority order
- 4.12 What to do about it?
- 4.13 Suggested Heuristics
- 4.14 Zero-overhead rule

5 Post-presentation discussion (Craig)

- 5.1 Heuristics and Design Decisions
- 5.2 Possible courses of action?
- 5.3 What is it for?
- 5.4 Can the features be adapted?
- 5.5 Costs and zero overhead
- 5.6 Characterization of features
- 5.7 Customization points
- 5.8 Embedded C++
- 5.9 Zero overhead exceptions (P0709)
- 5.10 Order of attack
- 5.11 Existing papers
- 5.12 Strict Aliasing

6 Presentation material from Kirk Shoop

- 6.1 Why am I here?
- 6.2 Freestanding (by scanning P1376R0)
- 6.3 Iterators
- 6.4 Machines over time
- 6.5 a single uniform abstract machine...
- 6.6 Proposal
- 6.7 Bonus

7 Post-presentation discussion (Shoop)

7.1 Concrete cases

7.2 Feature Test Macros

8 Lower level library

8.1 Graphics

8.2 Guidelines

9 Going Forward

9.1 Constexpr

9.2 General alternatives

9.3 Other papers?

§ 1. Summary

We discussed the target environments for freestanding (kernel environments, small microcontrollers, and GPUs). We discussed the problematic features that would need to be addressed in order to get a useful lowest common denominator subset. We talked about using concepts and subsumption to address the freestanding library.

§ 2. Opening commentary on subsetting

Ben Craig: ABQ poll was in favor of getting rid of `<typeinfo>` and `<exception>` header

Jan Wilmans: What was the opposition against getting rid of typeid? Were those reasons recorded?

Paul McKenney: Some people are passionate about having programs that can run everywhere, but portability has to be limited if we are going to make progress.

Hubert Tong: Might want to draw prior art from video encoding profiles [Editor's note: likely referring to H.264 profiles]

Ben Craig: POSIX, and I think Vulkan have feature sets?

Michael Wong: You mean OpenGL?

Ben Craig: Unsure, it was discussed some in SG1 though.

Michael Wong: Some discussion about flexible profiles in the OpenGL standards.

§ 3. Lead in to presentation

Michael Wong: Unclear what definition of freestanding and embedded are. Is it time to remove the mention of freestanding from C++?

Bjarne: Don't think the main problem with subsetting is portability. There isn't one thing / subset that serves a large enough group to be more dominant than other subsets. Some environments can't use malloc, but can use exceptions. Some machines with no floating point, but have malloc. The problem is multidimensional. Limited space is one concern, hard real time constraints as another, also other dimensions.

Ben Craig: I have some slides addressing those questions.

§ 4. Presentation material from Ben Craig

§ 4.1. "What is embedded?" is the wrong question.

Embedded is a large space, some care about safety, real time, limited space, absence of a monitor.

Not useful for the context of this discussion.

Freestanding isn't just trying to hit embedded use cases, like kernels / drivers and GPUs. Drivers have lots of resources, but they don't control the entire world.

§ 4.2. "What is an OS?" not useful either.

Does a scheduler count?

Could be an OS that has never left my organization.

Low feature set OSes may still want freestanding.

Scheduler could be in silicon.

If I'm inside the OS, do I have the benefits of an OS?

§ 4.3. What is a standard?

[A standard] describes not only what is "legal" source text, *but also what a programmer can rely on in general* and what is implementation defined

-Bjarne Stroustrup [D&E]

When starting freestanding investigations, I found the list of features that were troublesome to use as a customer because they had partial or no implementations in the environments I was working on. I started from the list of things that didn't work. Some items were well advertised (like exceptions and RTTI), but no advertising about others, like how virtual destructors use operator delete.

§ 4.4. Conforming C++ hosted toolchain experience

Compare / contrast with freestanding

Lots of things you can do everywhere.

You can expect everything to be there, if not today, then soon.

Libraries can rely on everything being present.

You may not be able to use any given library, as that library may use more resources than your machine has, or the library may not meet other requirements.

Generally, the out of box experience is good, and all the features are present without significant user intervention required.

§ 4.5. Hosted C++: The dark side

Lots of things you can't do portably in C++.

As community comes up with good abstractions we standardize them.

Everyone uses their own dialect of the language, using hosted + extensions.

People work through dialect variance with the preprocessor, build system, or by knowing what kind of system they are targeting.

This is particularly challenging with SIMD. Intrinsics aren't portable from processor to processor or compiler to compiler. Sometimes, you can even build and run things on one architecture, only to have

your program fail to run on a different, slightly older machine.

Michael Wong: We've been through a long history of different implementations of these, often with intrinsics or dialects. As these mature, we try to standardize these. SIMD is one of the leading ones, people working hard on heterogeneous and shared memory.

Ben Craig: Yes. As we figure out what works, be standardized, and have vendor buy in, we standardize them. As we come up with technical solutions, we incorporate them.

§ 4.6. Freestanding C++17 experience: Theory

Lots of things you can do entirely in conforming freestanding C++

- Math!
- Custom data structures!

The C++ facilities can generally be assumed to be “everywhere”

Libraries can design around all the facilities being present

§ 4.7. Freestanding C++17 experience: Practice

If you want a conforming implementation, then you need to finish the job of the toolchain.

Toolchain vendors don't know how your environment spells “malloc”

Someone has to call global constructors, and the toolchain vendors don't control the entry point

Someone has to do the loader + toolchain + threading hand shake to get thread local storage to work

Need to wire up exception tables...

Everyone uses their own dialect of the language (freestanding – conforming features + varying extensions)

People work through dialect variance with the preprocessor, build system, or by knowing what kind of system they are targeting

Michael Wong: With hosted, people add to existing base. With freestanding, people subtract, then add some.

Ben Craig: Right

§ 4.8. Freestanding C++ experience: Dark Side

Integer Math, can't rely on floating point

Custom heap-free data structures

Can still use new and delete with custom allocators.

The C++ facilities can't generally be assumed to be "everywhere"

20+ years on, and we still can't assume that exceptions will be enabled

No reason to expect the next 20 will be any different

Libraries can't design around all the facilities being present

Libraries that rely on the heap and exceptions are unusable in many environments

§ 4.9. Source of the bits?

Lots of vendors involved

Given a technology stack of ...

- Tensilica (chip vendor)
- FreeRTOS
- GCC
- Customer application

... who should be providing the core language C++ runtime support?

Are we going to expect GCC to know about FreeRTOS and every other RTOS so that it can provide support?

Are we going to expect FreeRTOS to provide special support for C++ and every other systems programming language?

Should Tensilica do that because they own the ABI and startup scripts?

In practice, it's the customer application that does that, especially if they want to be conforming.

Michael Wong: I see this change even more as we move to crazy kinds of chips, like for tensor processing. Maybe we are back to where we used to be, each group has to build their own compiler to support their technology that they can.

Ben Craig: and each company and each application does a different amount of that, making it very difficult to share code across organizations.

§ 4.10. Freestanding Goal

The freestanding C++ facilities can generally be assumed to be “everywhere”

- If not today, then “soon”, as toolchains catch up to the spec
 - So long as we have a reasonable expectation that they can / will catch up.

Libraries can design around all the facilities being present

- Including the standard library

Using the conforming C++ facilities works easily “out of the box”

- Should not require the user to implement an OS to support freestanding

Don't want to require customers to write an OS that looks like posix to support GCC / Clang, or require users to write an OS that looks like Windows to support MSVC

§ 4.11. Problem features in priority order

- HIGHEST
- Exceptions
- Default operator new & delete
- virtual destructors
- thread_local
- RTTI
- Global init and teardown
- atexit & friends

- Locked atomics
- Thread safe statics
- Floating point
- LOWEST

Priority based off of how difficult they are to incorporate, as well as how much they influence library design.

Exceptions influence library design a lot, floating point doesn't influence non-math library design much.

I would be thrilled if we addressed all the features except for floating point.

Exceptions called out again, because they use the heap (on Itanium ABI), usually involves a type with virtual destructor. Need storage that is local to a thread for `uncaught_exceptions` and `make_exception_ptr`, even if that storage isn't spelled `thread_local`. Needs a lot of the parts of RTTI. When things go bad, you need terminate, which falls under `atexit` & friends. If you want to address any of those pieces (like RTTI), then that implies doing something about exceptions as well.

Don't need to get rid of problem features, just need to do something about them.

§ 4.12. What to do about it?

Accept non-compliance

- This is the status quo

Change the feature

- Have to be sure not to fork the language further

Remove the feature

- Not ideal

Change the world

- Probably impossible

Pitch in one of my papers on changing when virtual destructors are ODR-used. This would allow them to stay in.

Don't like removing features, but don't see a lot of alternatives for some features.

Don't know how to change the world to make the problem features generally available.

§ 4.13. Suggested Heuristics

- Freestanding is signal / interrupt safe
- Freestanding requires no special dispensation from the operating environment above what freestanding C99 requires
- Freestanding strictly follows the zero overhead rule (you don't pay for what you don't use)

How did I decide what's in the library, what's in the language? So far that has been based off of experience. These heuristics are an attempt to distill that into bullet points. Possible you agree with some of these, or all of these. We can make designs based off of which of these we choose.

For example, if we say that freestanding is signal / interrupt safe, then we need to get rid of locked atomics.

If we don't take the signal / interrupt safe bullet, but we do keep the "no special dispensation from the operating environment" item, then we still need to get rid of exceptions.

§ 4.14. Zero-overhead rule

The zero-overhead rule is probably the one that has the sharpest edge when it comes to rejecting a suggested feature

-Bjarne Stroustrup [D&E]

Cutting features hurts, but non-zero overhead is larger than zero.

§ 5. Post-presentation discussion (Craig)

§ 5.1. Heuristics and Design Decisions

Michael Wong: So your suggested heuristics will affect design decisions, depending on which choices we make?

Ben Craig: Yes. In particular, SG1 wasn't interested in getting rid of locked atomics or thread-safe statics, given our current lack of direction. Part of the reasoning there is that it is possible to write a spin lock without an operating system. Spin locks aren't interrupt safe though, and some operating systems are basically all interrupts though. In those situations, we would have the facility, but you couldn't rely on it. That's why I want to cut it. However, if we get agreement on a direction though, I can drop those aspects. Thread-safe statics, locked atomics, and floating point aren't "load bearing" for the proposal.

§ 5.2. Possible courses of action?

Michael Wong: Assuming that "Removing Features" and "changing the world" are off the table?

Ben Craig: For removing a feature, current paper says not to provide a default operator new / delete, but users can still provide their own. Exceptions in my proposal are gutted, but some of the syntax is still there, leaving just a little to help with portability. Thread_local is gone. You can pick a direction (remove, change, ignore) on a per feature basis.

§ 5.3. What is it for?

Bjarne: I don't feel that I understand what this is for? How do you characterize the domain in which these things are useful or not useful? "I want" or "I don't want" are not arguments.

Ben Craig: These features are difficult to implement or use effectively in kernel environments, small microcontrollers, and GPUs. Today, these features do not work well in these environments, usually by failing to compile or link.

Bjarne: In kernel environments I've seen exceptions used well for speeding up things.

Ben Craig: This isn't trying to say that I don't want the features to exist there, this is more acknowledging that there are environments where people haven't figured out how to get these features, even when they control the entire technology stack. I'll pick on Microsoft for a moment. The kernel driver environment does not support C++ exceptions. Microsoft controls the compiler and the operating system. They don't control the chipset, but we know x86 and x64 processors. Microsoft has had 20 years to implement exceptions in the kernel. I have heard that many kernel driver developers would like exceptions to be there. Yet they are not there. That suggests to me that it is difficult or too expensive to implement exceptions in the kernel. In particular, exceptions want storage that is local to a thread, implementing that in a kernel environment would incur overhead on every thread in the system.

Niall Douglas (chat): Also Windows kernel DOES support win32 exceptions, so the lack of C++ exception support (which is just a special win32 exception) is particularly relevant.

§ 5.4. Can the features be adapted?

Michael Wong: I don't think this is saying we want to get rid of any of these features. Is there any way we can adapt them so that they can work in different environment's needs.

Ben Craig: With type based exceptions, I don't know of a way to get them such that you can throw an arbitrary type and catch that type, or something related to it, in a zero overhead way. I have heard of implementations that do something similar, but have various restrictions, such as single inheritance only allowed. Those implementations have low overhead in terms of, the frames that don't ever have an exception thrown through them, there is still space overhead on those. Those implementations exist, but it isn't zero overhead.

§ 5.5. Costs and zero overhead

Bjarne: When looking at zero overhead, you have to look at the cost of using error codes instead.

Ben Craig: The world that kernel and embedded development is in right now, we have to pay something for error handling. The amount we are paying for our error handling is proportional to the amount we use it. If I have a big call stack across multiple translation units and none of those frames do anything with an error (no failure cases), those frames, those functions, none of them have any overhead. They don't pay anything for error code handling. With exceptions, if you add a lot of frames that would be noexcept all the way down, if I don't mark them noexcept, I get overhead. Even if I do mark them all noexcept, I can still get some overhead.

Hubert Tong: Would a side stack that only grows during the unwinding be a cost that is considered unacceptable

Ben Craig: Not sure. Don't know enough about that approach to answer definitively. Part of zero overhead is also size. If you still need the exception tables, similar to what the current table based exception handling has, those tables are significant overhead. I don't know if the side stack approach solves that or not. Also not sure if `uncaught_exceptions` or `make_exception_ptr` can be supported on a multi core system without storage that is local to a thread.

Carter Edwards (chat): Another example: Managing `thread_local` does not scale well with 10k+ concurrency; e.g., GPU threads.

Jan Wilmans: Ben describes that the whole point is to describe a subset of C++ that you can rely on being available in any of the described environments.

Ben Craig: Right now, that's the description I'm liking.

Jan Wilmans: Bjarne said that it isn't clear what that environment is. What are we actually targetting? Part after that, exceptions is one of the major things blocking the usability of the standard library, and there are technical difficulties in getting that to work.

§ 5.6. Characterization of features

Hubert Tong: I believe that the claims that the environments we are targeting is not clear are valid, however, the identified features are all of the sort that requires side support which is extensive in implementation. I'm not certain that we are really talking about environment vs. implementation cost and overhead.

Jan Wilmans: Are you saying that the accurate description doesn't matter because of the overhead in size?

Hubert Tong: Not just in size, but also in the cost of the toolchain in terms of how much development has to go into it. If you have a bare metal system and you want C++ to run on it... what parts are you going to target first and what parts are going to be so cost prohibitive that if you don't need it you aren't going to use it?

Jan Wilmans: Getting rid of the overhead of every vendor needing to implement the same things again.

Ben Craig: And for increasingly exotic platforms.

Ben Craig: I'm not too worried about having GCC, Clang, Microsoft, or Intel doing a bit more work on their side. I am worried about having every customer implement these things.

Jan Wilmans: What may be helpful is to clarify to other groups that it isn't so much about defining the subset, but defining what needs to be standard implemented. May still mean it isn't available on some platform, but at least there is a standard implementation that any vendor can adopt.

Ben Craig: Not sure about that. If there were standard implementations for these things that were applicable everywhere, they wouldn't be features on the list. Most of these features require operating environment knowledge. There are too many environments and they are too obscure for the big compiler vendors to target.

Michael Wong: The reason these features are on the list is because they always require some cooperation with the local environment to operate efficiently. In some cases, they could be implemented if efficiency wasn't one of your biggest concerns.

§ 5.7. Customization points

Jan Wilmans: Does that mean that, aside from having to specify the set of available C++ features, we should also specify better customization points for those environments?

Ben Craig: There are two areas where I want to take advantage of customer supplied customization points. One of them is with operator new / delete. I don't want to supply those myself, but I do want to allow customers to provide them. The other is for terminate. There is already a customization point at runtime, but I want one at build time. In many exotic systems, the standard library doesn't know how to terminate the system, that is up to the customer to figure that out. For things like thread_local storage and thread safe statics, the protocol for the customization points would be very constraining. You could standardize the Itanium ABI, but then the Microsoft implementation would suffer. Even with some of these features, there is per-processor implementations. There are places where we can take the customization point approach, and I like those. Thread local and thread safe statics are hard though, and I haven't thought through those much.

Jan Wilmans: You're saying having customization points or not having them, won't always work. Have to specify that the feature will be there, but how the feature is implemented is not specified.

Ben Craig: That's what we do today. If you get much lower level than you end up constraining implementations.

Ben Craig: Are there features that are not on my list that should be? If there are features that should be on this list, that may mean that I have not found a lowest common denominator. There's at least one area that is not on the list that will be difficult to get on the list (code size).

§ 5.8. Embedded C++

Bjarne: Somebody tried doing this in the 90s and it was called Embedded C++. One of the things we found that time was that it was not well characterized what it was supposed to do. I was particularly amused that namespaces were banned.

Ben Craig: That seemed strange to me as well.

Bjarne: The feature set seemed somewhat random. You're less random than they were. I was particularly amused that namespaces were banned. The point is, unless you characterize rather precisely the domain you are trying to solve and what the solution and scope of solution, you risk falling into the Embedded C++ trap.

Ben Craig: Some of the things I am doing to try to avoid the Embedded C++ trap is to not focus on limiting implementer burden.

Bjarne: It was definitely trying to limit implementer burdens. They also thought they were simplifying the language to make it easier for developers.

Ben Craig: Other thing I'm doing is trying to stick to a strict subset and not a fork. Embedded C++ changed some semantics, and I'm really trying hard to avoid that.

§ 5.9. Zero overhead exceptions (P0709)

Bjarne: Problem is when you take away features, you find that you need something else. That's a hard problem.

Ben Craig: I agree, the advantage we have here is that we have code in pseudo-C++ that are already operating under these constraints. None of these features are enabled by default in the Windows kernel. There are several companies that are writing C++ drivers in the kernel. Yes, there is a gap in error handling and it makes constructors and overloaded operators difficult to use. I look forward to solutions to that, particularly in Herb's zero cost exceptions [Editor's note: P0709]. I will say they aren't strictly necessary, as we have gotten by without them for 20 years.

Bjarne: Microsoft currently has 3 kinds of exceptions. If you add in Herb's proposal, you would have 4. The old ones won't go away.

Ben Craig: I agree.

§ 5.10. Order of attack

Jan Wilmans: Not sure if this is acceptable, but would it be possible to pick the low hanging fruit first? The atexit and global teardown, new / delete, virtual destructors. That all seems doable, maybe exceptions. The thread_local and RTTI, locked atomics, thread safe statics, that all sounds more difficult. Would it make sense to sequence this in steps.

Ben Craig: I am hoping to get direction at some point. For the direction to be most useful, a big picture view needs to be given. Once we have direction, I think it will make the most sense to split P1105 into

smaller papers, and those papers can progress. I want to get general wg21 agreement on where we are going first. I've done things backwards too long on this.

Michael Wong: I'm hesitant on taking a vote at this point. I need people to think about this and absorb it more. Also, not to reverse the process, but have some directions at a high level as to where we are going.

Carter Edwards (chat): Suggest framing in terms of "scaling" considerations: scaling down processor complexity, scaling down runtime support, scaling up concurrency, etc

§ 5.11. Existing papers

Paul Bendixen: One of the possible directions you outlined earlier was that you made one paper that adds to freestanding and one that subtracts. Could this be a possible direction to go in order to get more familiar with what it would mean to have an actual freestanding implementation before we settle on a final subset or definition.

Michael Wong: Is that what you have Ben?

Ben Craig: Yes. I have one paper P0829, Freestanding Proposal, that is strictly additive and is for the library. My "No Lower Level" paper, P1105, subtracts features and is mostly a core language paper, with a little bit of library support.

Michael Wong: We will keep that in mind during our potential direction discussions.

Charley Bay (chat): FYI, on Freestanding: IMHO Ben's proposal is very well bounded and described, and reflects *real-world* practice. A "freestanding" exists in practice -- this is called C. It is unfortunate that an "all-of-noting C++" means we cannot use C++ in some environments.

§ 5.12. Strict Aliasing

Niall Douglas (chat): Even in C, not all of C is provided by many embedded toolchains. A recent, slightly angry, debate on WG14 was over strict aliasing for example (some feel very strongly that it needs to be banned in future C standards).

Paul E. McKenney (chat): As in prohibiting the `-fno-strict-aliasing` compiler flag? Or as in requiring C to always act as if `-fno-strict-aliasing` was set?

Niall (chat): As in requiring C to always act as if `-fno-strict-aliasing` was set. So, strict aliasing would be specifically banned by a future C standard. WG14 voted to reject the proposal, incidentally, but it

wasn't unanimous.

§ 6. Presentation material from Kirk Shoop

subsumption in the abstract machine(s) freestanding without subsetting

§ 6.1. Why am I here?

- discussion of affinity
- consider changes to the abstract machine
 - to deal with some of the heterogenous machines
- perhaps a subsumption hierarchy, similar to iterators and sender / receiver might provide a framework in which these machines could be defined.

§ 6.2. Freestanding (by scanning P1376R0)

- `std::library` definition
- not a subset
- scope varies on multiple axis
 - hardware features
 - language features
 - toolchain features

This felt similar to iterators

§ 6.3. Iterators

- defined in the `std::` library
- not a subset
- scope varies on multiple axis
 - input

- output
- traversal
- position

We have algorithms that state that I require an input iterator, or at least a forward iterator, etc. We have dispatching underneath the algorithm that can do a different implementation for a random access iterator. None of that is subsetting, it is just capabilities dependent. If you try to pass in an iterator that doesn't support the minimum for an algorithm, it won't compile.

§ 6.4. Machines over time

- then
 - diversity of discrete machines
 - abstract machine attempts to be portable over these discrete machine
- now
 - diversity of connected machines
 - our computer contains (arguably) multiple machines. They are connected to each other, share some resources, they have different instruction sets and capabilities.

§ 6.5. a single uniform abstract machine...

Machine definition

- one definition to rule them all
 - create the illusion of homogeneity

GPU accelerators try to unify the memory models so that programmers don't notice the paging in and out across the gpu to the host to keep the idea that there is a shared memory resource.

- a collection of connected machines
 - define each machine
 - maybe even to each core of a processor
 - define the connections separately

§ 6.6. Proposal

"Always start with the algorithms" - Alex Stepanov via Eric Niebler ...or in this case, the library

- follow the pattern used to define Concepts
 - for every library feature define the feature-set that it requires
 - organize those features into hierarchies of concepts
 - define the abstract machine as some permutation of supported Concepts
 - implement the library with requirements specified via Concepts
 - implement the library with tag-dispatch, based on the concepts
 - implement the library with awareness of the connections between machines
 - for things like mutexes and condition variables and things that are intended to coordinate or communicate between regions

§ 6.7. Bonus

constexpr

- constexpr defines an abstract machine
- constexpr can be exposed as an eval method in std::
- the compiler can call eval in std:: to process constexpr

distributed

- a collection of machines with connections
 - your standard library would still be one whole standard library, but it would know from the current context whether it was compatible or not.

§ 7. Post-presentation discussion (Shoop)

Hubert Tong: Current implementation practice around turning off exceptions is to keep the same interfaces, they just work differently. That makes it difficult to have the code compile where it can and error where it can't.

Kirk Shoop: I think this would be something where you would try to define exceptions as a subsumption hierarchy. Flying by the seat of my pants, at the bottom would be the least restrictive requirement, for example, that the function is exception transparent. Another would be that a function depended on exceptions in order to run, and another would be that in the presence of exceptions, the function terminates. If you know for a given library feature what it requires, then you can have the compiler declare what the current mode is through these concepts so that the library doesn't compile unless it can support the current context.

Hubert Tong: Sure, that makes sense. I'm just trying to say that if we go in this direction, then people with existing code will need to choose alternative interfaces and we will need to standardize library interfaces that provide alternatives in the case where you don't need / want the exceptions.

Kirk Shoop: I think that may very well happen, that over time we say that we still need this functionality with fewer requirements, so we are going to need to come up with another shape for it that matches these requirements.

Bjarne: The filesystem library has already gone this way.

Bjarne: I like some of this. I think people have implemented the STL based on memory pools instead of a general allocator. There are a few things you cannot do, but most of the things you can do and you maintain the interface.

Ben Craig: For the stuff that is in templates, I think you can do a lot of this subsumption. There are some things in the standard library that are not in templates, and that is harder to do this kind of dispatching with because your `static_asserts` fire all of the time. That code may be built into a shared library somewhere as opposed to into your binary.

Kirk Shoop: I hope that static if would help with that particular one.

Ben Craig: I think static-if is pretty much dead. `constexpr` if only discards statements when you are in templates. There would need to be work done to make all of that go.

§ 7.1. Concrete cases

Ben Craig: I can give you a few concrete cases to stew on. There are three algorithms in the traditional, non-parallel algorithms header where, if you can get a temporary buffer, they have a different complexity guarantee than when they can successfully allocate a temporary buffer; `stable_sort`, `stable_partition`, and `inplace_merge`. So those are algorithms where you could consider what the dispatching is, and whether that is a desirable thing or not, to change the constant overhead and the big-O overhead based off of tag dispatching.

Kirk Shoop: That's what we do when we go between forward and random access for some algorithms as well

Ben Craig: Agreed

Ben Craig: Weird, in-between case that I've been forced to deal with in the freestanding proposal. When you call `std::visit` on a variant that is in the valueless by exception state, `std::visit` will throw. However, if exceptions aren't on in your program, you can't get in the valueless by exception state. This would be a case where this dispatching would avoid having a throw instantiated in a template function. Those are beneficial concrete places to think through.

Michael Wong: Do people like some of this, or not like some of this?

Jon McFarlane: This is an interesting direction. I'd like to see some examples even though this is preliminary and high level. Some more concrete examples would help paint a picture for me.

Ben Craig: Containers in particular would be a useful place. Show what vector and `unordered_map` would look like. Note that `unordered_map` normally depends on floating point.

Bjarne: I think concrete examples aren't just useful, they are essential.

Kirk Shoop: I agree with the approach that you start with the library and see what it depends on, then define these concepts by poring over and finding these dependencies and organizing them.

§ 7.2. Feature Test Macros

Paul Bendixen: Would these concepts be based on feature test macros? Would they need to be defined for each machine type that you are going to use.

Kirk Shoop: That is what makes sense to me.

Bjarne: I hope not. We now have 2^{100} dialects from those macros.

Kirk Shoop: Are those dialects or just traits?

Bjarne: oh yes. I can write my program to see what my compiler supports. I'm not saying that there are actually 2^{100} different settings for the compilers, but that's what I would have to assume. It's only manageable because you ignore most of them most of the time.

Kirk Shoop: I agree. I expect most of the complexity to be in libraries.

Jon McFarlane: I've found they are generally useless in an application, because you generally know what your target platforms are. When you are writing a library, you don't know all the features when writing your library because it is used in multiple applications. They provide a way to additively provide some features for the people that know they have those features available. They are practical in that respect.

Kirk Shoop: Most libraries wouldn't care about most of the features.

Jon McFarlane: I can add deduction guides if deduction guides are available.

Michael Wong: Bjarne, you said you don't think they should be feature test macros?

Bjarne: I would say feature test macros is always my last alternative. We started talking about iterators, that is done without feature test macros. One of the reason I don't like macros is that they foul up IDEs. What I see and what the compiler sees are two different things. If you use them, use them deep in the library, but don't make them the primary user interface.

§ 8. Lower level library

Michael Wong: Our last discussion is whether there should be a lower layer for C++ that is closer to the metal for C++ for libraries. This was a question that was posted to the forums coming out of San Diego. Charley and Arthur had laid out a few good points that should be captured. Leaving no room for a lower level language below C++, that is status quo from the current C++ standard as stated in Design and Evolution. Leaving no room for a lower level library below std library, this is not clearly said anywhere, and maybe it is something we should think about proposing. Some of the things the direction group has thought of as a test is that this design philosophy should only apply to foundational libraries, like `std::variant`, but not necessarily to all libraries.

Ben Craig: There are two closely related design goals. You mentioned the "No lower level language" rule, but there is also the "zero overhead" rule, where you don't pay for what you don't use. On the library side, we should stick with the zero overhead rule. For the most part, we do. There are a few place where we have standard library mandated globals, and those are places where I think we violate that a bit. There are some things getting kicked around in SG1 where there would be globals in the standard library that I would have to pay for if I accidentally include the header. So it's a related thing, but not the same.

Bjarne: I'm no fan of globals, but I don't think that's what the direction group was thinking about. Foundation libraries definitely follow the zero overhead principle. It doesn't mean there's no overhead, it means there's no overhead given what you are doing. That doesn't mean we couldn't have standard libraries that doesn't follow it. Why couldn't we have a convenience library for some groups of users

that are not really interested in zero overhead all the time. I don't think we have a decision that says the standard library should only provide foundational libraries for building more code and not convenience libraries.

Ben Craig: If I'm using it, I am great paying the overhead, even if it uses globals and it has some overhead and it isn't the fastest. If it's convenient then that is fine, as long as I'm actually using it and I'm only paying for it if I use it.

§ 8.1. Graphics

Bjarne: Let's take a concrete example. I feel some pain because I can't do a simple output of a histogram out of a piece of code that is gathering some data, because there is no standard graphics library. Now, the standard graphics library that would allow me to do this as a naive user would definitely not be zero overhead for experts. The point is that I don't see why we should only have one kind of graphics. One that's convenient and universally acceptable, and one that's specialized and takes advantage of the hardware.

Michael Wong: Nothing says that we can't have multiple kinds of graphics libraries.

Bjarne: One is to make the next Matrix movie, and the other is to allow me to write the histogram. I could do that in 1981 because it was part of the first library that ever existed for C++, but it most certainly isn't up to modern standards. I think that anything that is available universally would not be optimal to make the next Matrix movie. I'm not sure what this has to do with embedded systems or freestanding systems, it's just a comment on defining what we can have. You have to be careful when you say zero overhead or lower language underneath. It doesn't mean that every single line of code that we produce is optimal.

Ben Craig: I'll post a quick strawman for graphics. Let's say the graphics proposal has some globals (no idea if it does or not). I think I'm fine with that, so long as they are in graphics specific headers. If you were to do something incredibly silly, and include those headers from the `type_traits` header, then suddenly, the program that I thought was just doing compile time stuff is now constructing those globals before program start. As long as it is reasonable to make sure that I only pay for it if I'm actually using it by putting it in its own headers.

Bjarne: Agreed, agreed, agreed. We can't outlaw stupidity, but we can try to avoid it.

§ 8.2. Guidelines

Ben Craig: Niall had some guidelines at one point for things interacting with the OS. Something along the lines of preserve the same guarantees. That seems like the closest phrasing description to zero overhead when it comes to the library that I've heard. That's just from my recollection.

Niall: We came up with a set of guidelines. Second time it came to SG14, it was felt that it should turn into some core C++ guidelines instead. I haven't had the time to work on that.

§ 9. Going Forward

Micheal Wong: What can we do going forward with freestanding and hosted?

Charley Bay: I have a quick comment. This is really an eminently practical approach that reflects real world practice. I think Ben's slide on the problem features, that's the list. It's a pretty universal list, it shows up everywhere. A lot of people deal with real engineering constraints, footprint, toolchain, performance, latency, it's something, but those issues with real constraints go straight to Ben's list. So it's a very practical proposal that I think would be well received. Either those features cannot be implemented, or they are cost prohibitive either on implementation or runtime, or footprint or something. I'm working at a company, a lot of ones before, where we have a freestanding approach, it's called C. It's a hybrid code environment, that's on the C side, that's on the C++ side, and it's entirely because of Ben's list. Ideally it would be C++ because there are so many optimizations and advantages there, but due to the size of the team, there is concern over the execution discipline over time by different teams of different skill sets, and because that execution discipline by the developer cannot be guaranteed, the codebase will remain C. I just want to point out that Ben's approach is not an academic approach, it's a practical real world thing that would make a big splash in my opinion.

§ 9.1. Constexpr

Niall: There was a recent discussion on WG14 reflector. Historically there have been many safe C subdialect, and there is ongoing work on formally proving the C abstract machine. If we could get constexpr so that it could run most C code, we would be able to define the safe subset of C as that which can run under constexpr. How much of freestanding is implementable in constexpr, and is that a useful method of thinking about the problem.

Ben Craig: Floating point is the main thing that is contentious between the two. You can do floating point in constexpr, but not in freestanding. Note that floating point was also the lowest priority item on

my list. Also, you can do undefined behavior in freestanding, but not in `constexpr`. [Editor's note: usage of `new` and `delete` in `constexpr` is a newer point of contention.]

Niall: That's kind of where I was getting to. You get some floating point in `constexpr`, it may not be the floating point you were expecting. I was more referring to the UB aspect of this. Something where the compiler tells you that you are doing something bad before you even compile anything would be a good thing to have. Both for freestanding and everything else.

§ 9.2. General alternatives

Bjarne: I would like to point out in this practical real world vs. academic. I remember when `constexpr` was considered not only useless, but unimplementable and horrible academic version. We have to be a little idealistic to change the world, sometimes we can change it for the better.

Niall: I'll suggest an alternative way of looking at freestanding. Freestanding can be something that can be applied to embedded systems, and that has a lot of value and is worth discussing. Another way to look at it is to draw a line between the things that are universally useful everywhere, and the things that are only useful sometimes. I think that dividing line can be much more useful in terms of direction and defining the correct form of the language and library, rather than something that can be delivered into a standard.

Michael Wong: So more like a standing document or guidance?

Niall: I spend a lot of time reading the wg21 papers that come out and emailing people privately and telling people that if they split their library "here" that you get a much better proposal. A lot of the time it is based on splitting out the abstract from the particular. That is something that the C++ standard has not been as disciplined about historically, and I think it could get a better. Then we wouldn't have `thread_local` and GPUs for example.

Ben Craig: On the core language side, one of the reasons I am bothering pushing for this at all is because it feeds so much into the library design. If we only got a standing document of things that should be avoided if they are not integral to the design, then that wouldn't be ideal, but so long as we can still make library decisions based off of it, that would still be useful. Not as useful as if I got everything I wanted, but it would still be something.

§ 9.3. Other papers?

Michael Wong: Kirk has put forward a proposal, and Ben is looking for someone else to help push this forward. Is Kirk interested in writing what he has? As another way of pushing forward? For using

Concepts for freestanding? Would that serve what the group would want to do? Freestanding is not necessarily a one day job or a couple of meetings job. People have to slowly get used to it, that we are trying to make this change and that there are multiple proposals going towards this change.

Ben Craig: I know that JF Bastien, Loius Dionne, and Bruno Cardosa Lopes are working on freestanding things in a slightly different direction, even though they haven't had an overarching paper at this point.

Michael Wong: Have they put this paper out before or is this something they are planning for?

Ben Craig: They have all been coauthors on papers, one of them dealing with global destructors, one dealing with module level attributes. One other in the freestanding space. [Editor's note: P1246, "The no_float function attribute"]

Michael Wong: Maybe we can help them move forward. I'm sure they would come on this call and give us some ideas.

???: All the papers on constexpr have a multi-release arch going to them. I feel that something like the machine concepts idea could be a way of starting our own arch for things like the affinities and stack sizes and things that have been on our wish list since year -1 essentially.

Ben Craig: I think that's possible, P0829 may be the lowest level of all those concepts, and then we start putting things on top of it.

Paul Bendixen: I would encourage Kirk to write a paper. I still haven't seen how to make this concept based approach work. As far as I can tell, concepts need to be more-or-less, syntax sugar for templates. If you want to have your algorithm decide upon your concept, you need to pass it in somehow.

Hubert Tong: Yes, concepts need something to pass in, but at the same time, you could have a standard library trait. Say that there is a trait that gives you a type, and if the type models whatever concept, and the type is a representation of your system or environment. You could ask for that and pass it in to the concept when your library needs to choose what implementation to use.